NETWORKS LAB PRACTICAL

20 Topics — Complete Server, Client and SimPy Code

Prepared for practical revision. Each topic has separate runnable files for socket communication and a SimPy educational simulation.

Run instructions: Open two terminals for server/client. Run `python server.py` first, then `python client.py`. For simulations install SimPy with `python -m pip install simpy` and run `python simpy_simulation.py`.

Accuracy note: All Python files were syntax-checked. Socket demos use localhost and are designed for same-computer testing. Some simulations intentionally simplify complex protocols; notes identify these limitations. A TCP/UDP wrapper is not itself a full implementation of ARQ, CSMA, CDMA, or real routing.

01_TCP — TCP request-response

server.py

Run server first. This is a TCP request-response example.

```
import socket

HOST = "127.0.0.1"
PORT = 6201

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(5)
    print(f"TCP server listening on {HOST}:{PORT}")
    while True:
        conn, addr = server.accept()
        with conn:
            print("Connected:", addr)
            data = conn.recv(1024)
            if not data:
                continue
            message = data.decode()
            print("Client:", message)
            conn.sendall(("Received: " + message).encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6201

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    message = input("Enter message: ")
    client.sendall(message.encode())
    reply = client.recv(1024).decode()
    print("Server:", reply)
```

simpy_simulation.py

SimPy models request service time; it is not a simulation of TCP's internal protocol.

```
import simpy

def tcp_request(env, name, server, duration):
    start = env.now
    print(f"{env.now:.1f}: {name} sends request")
    with server.request() as req:
        yield req
        print(f"{env.now:.1f}: {name} begins service")
        yield env.timeout(duration)
        print(f"{env.now:.1f}: {name} receives response")
    print(f"{name} response time: {env.now - start:.1f}")

env = simpy.Environment()
server = simpy.Resource(env, capacity=1)
for i, duration in enumerate([2, 1, 3], start=1):
    env.process(tcp_request(env, f"Client-{i}", server, duration))
env.run()
```

02_UDP — UDP request-response

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6202

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind((HOST, PORT))
    print(f"UDP server listening on {HOST}:{PORT}")
    while True:
        data, addr = server.recvfrom(1024)
        message = data.decode()
        print("Client:", addr, message)
        server.sendto(("Received: " + message).encode(), addr)
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6202

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.settimeout(5)
    message = input("Enter message: ")
    client.sendto(message.encode(), (HOST, PORT))
    try:
        data, _ = client.recvfrom(1024)
        print("Server:", data.decode())
    except socket.timeout:
        print("No reply: request timed out")
```

simpy_simulation.py

```
import simpy
import random

random.seed(10)
def packet(env, packet_id, channel, loss_probability=0.1):
    yield env.timeout(random.uniform(0.1, 1.0))
    with channel.request() as req:
        yield req
        print(f"{env.now:.2f}: Packet {packet_id} sent")
        yield env.timeout(0.2)
    if random.random() < loss_probability:
        print(f"{env.now:.2f}: Packet {packet_id} lost")
    else:
        print(f"{env.now:.2f}: Packet {packet_id} delivered")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
for i in range(1, 11):
    env.process(packet(env, i, channel))
env.run()
```

03_TCP_Chat — TCP chat

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6203

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen(1)
    print("TCP chat server waiting...")
    conn, addr = server.accept()
    with conn:
        print("Connected:", addr, "(type /quit to exit)")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode().strip()
            print("Client:", message)
            if message == "/quit":
                break
            reply = input("You: ")
            conn.sendall(reply.encode())
            if reply.strip() == "/quit":
                break
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6203

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    print("Connected. Type /quit to exit.")
    while True:
        message = input("You: ")
        client.sendall(message.encode())
        if message.strip() == "/quit":
            break
        data = client.recv(1024)
        if not data:
            break
        reply = data.decode()
        print("Server:", reply)
        if reply.strip() == "/quit":
            break
```

simpy_simulation.py

```
import simpy

def chat_exchange(env, user, channel):
    for message_no in range(1, 4):
        with channel.request() as req:
            yield req
            print(f"{env.now:.1f}: {user} sends message {message_no}")
            yield env.timeout(0.5)
            print(f"{env.now:.1f}: {user} receives reply {message_no}")
        yield env.timeout(0.2)

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(chat_exchange(env, "Alice", channel))
env.process(chat_exchange(env, "Bob", channel))
env.run()
```

04_UDP_Chat — UDP chat

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6204

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind((HOST, PORT))
    print("UDP chat server ready")
    while True:
        data, addr = server.recvfrom(1024)
        message = data.decode().strip()
        print(f"Client {addr}: {message}")
        if message == "/quit":
            server.sendto(b"Goodbye", addr)
            continue
        reply = input("You: ")
        server.sendto(reply.encode(), addr)
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6204

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.settimeout(10)
    print("UDP chat: type /quit to exit")
    while True:
        message = input("You: ")
        client.sendto(message.encode(), (HOST, PORT))
        try:
            data, _ = client.recvfrom(1024)
            print("Server:", data.decode())
        except socket.timeout:
            print("Reply timed out")
        if message.strip() == "/quit":
            break
```

simpy_simulation.py

```
import simpy
import random

random.seed(11)
def udp_message(env, node, channel):
    yield env.timeout(random.uniform(0, 2))
    with channel.request() as req:
        yield req
        print(f"{env.now:.1f}: {node} sends datagram")
        yield env.timeout(0.3)
    print(f"{env.now:.1f}: {node} datagram sent (delivery not guaranteed)")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
for i in range(1, 5):
    env.process(udp_message(env, f"Node-{i}", channel))
env.run()
```

05_Multi_Client_Chat — Multi-client TCP chat

server.py

```
import socket
import threading

HOST = "127.0.0.1"
PORT = 6205
clients = []
clients_lock = threading.Lock()

def broadcast(message, sender):
    with clients_lock:
        recipients = list(clients)
    for client in recipients:
        if client is not sender:
            try:
                client.sendall(message)
            except OSError:
                remove_client(client)

def remove_client(client):
    with clients_lock:
        if client in clients:
            clients.remove(client)
    try:
        client.close()
    except OSError:
        pass

def handle_client(conn, addr):
    print("Connected:", addr)
    try:
        conn.sendall(b"Welcome to multi-client chat. Type /quit to leave.")
        while True:
            data = conn.recv(1024)
            if not data:
                break
            message = data.decode().strip()
            if message == "/quit":
                break
            broadcast(f"{addr[0]}:{addr[1]}: {message}".encode(), conn)
    except OSError:
        pass
    finally:
        remove_client(conn)
        broadcast(f"Client {addr[1]} left.".encode(), conn)
        print("Disconnected:", addr)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print(f"Multi-client chat on {HOST}:{PORT}")
    while True:
        conn, addr = server.accept()
        with clients_lock:
            clients.append(conn)
        threading.Thread(target=handle_client, args=(conn, addr), daemon=True).start()
```

client.py

```
import socket
import threading

HOST = "127.0.0.1"
PORT = 6205

def receive(sock):
    while True:
        try:
            data = sock.recv(1024)
            if not data:
                print("\nServer disconnected")
                break
            print("\n" + data.decode() + "\nYou: ", end="")
        except OSError:
            break

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    print(client.recv(1024).decode())
    threading.Thread(target=receive, args=(client,), daemon=True).start()
    try:
        while True:
            message = input("You: ")
            client.sendall(message.encode())
            if message.strip() == "/quit":
                break
    except (OSError, KeyboardInterrupt):
        pass
```

simpy_simulation.py

```
import simpy
import random

random.seed(12)
def client_request(env, name, server):
    for i in range(2):
```

```
        start = env.now
        with server.request() as req:
            yield req
            print(f"{env.now:.1f}: {name} sends chat message {i+1}")
            yield env.timeout(random.uniform(0.2, 0.8))
        print(f"{env.now:.1f}: {name} message handled; wait={env.now-start:.1f}")
        yield env.timeout(random.uniform(0.1, 1.0))

env = simpy.Environment()
server = simpy.Resource(env, capacity=2)
for i in range(1, 6):
    env.process(client_request(env, f"Client-{i}", server))
env.run()
```

06_ARP — ARP lookup

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6206
arp_table = {
    "00:11:22:33:44:55": "192.168.1.10",
    "AA:BB:CC:DD:EE:FF": "192.168.1.20",
    "10:20:30:40:50:60": "192.168.1.30",
}

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("ARP demo server started")
    while True:
        conn, addr = server.accept()
        with conn:
            mac = conn.recv(1024).decode().strip().upper()
            ip = arp_table.get(mac, "MAC_NOT_FOUND")
            conn.sendall(ip.encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6206

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    mac = input("Enter MAC address: ").strip().upper()
    client.sendall(mac.encode())
    print("ARP demo reply IP:", client.recv(1024).decode())
```

simpy_simulation.py

Educational ARP table, not the OS ARP cache or a real Ethernet ARP broadcast.

```
import simpy

arp_table = {"AA:BB:CC:DD:EE:FF": "192.168.1.20"}

def arp_request(env, node, mac, channel):
    start = env.now
    with channel.request() as req:
        yield req
        print(f"{env.now:.1f}: {node} broadcasts ARP request for {mac}")
        yield env.timeout(0.5)
        print(f"{env.now:.1f}: ARP reply = {arp_table.get(mac, 'NOT_FOUND')}")
    print(f"Lookup time: {env.now-start:.1f}")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(arp_request(env, "Host-A", "AA:BB:CC:DD:EE:FF", channel))
env.process(arp_request(env, "Host-B", "00:00:00:00:00:00", channel))
env.run()
```

07_DNS — DNS lookup

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6207
dns_table = {
    "example.com": "93.184.216.34",
    "college.edu": "192.0.2.10",
    "localhost": "127.0.0.1",
}

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("DNS demo server started")
    while True:
        conn, addr = server.accept()
        with conn:
            domain = conn.recv(1024).decode().strip().lower()
            answer = dns_table.get(domain, "DOMAIN_NOT_FOUND")
            conn.sendall(answer.encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6207

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    domain = input("Enter domain name: ").strip().lower()
    client.sendall(domain.encode())
    print("Resolved address:", client.recv(1024).decode())
```

simpy_simulation.py

This is a local dictionary-based DNS demonstration, not full recursive DNS.

```
import simpy

def dns_lookup(env, domain, delay, server):
    start = env.now
    with server.request() as req:
        yield req
        print(f"{env.now:.1f}: Resolving {domain}")
        yield env.timeout(delay)
        print(f"{env.now:.1f}: {domain} lookup completed")
    print("Response time:", env.now-start)

env = simpy.Environment()
dns_server = simpy.Resource(env, capacity=1)
for domain, delay in [("a.example", 0.5), ("b.example", 1.0), ("c.example", 0.3)]:
    env.process(dns_lookup(env, domain, delay, dns_server))
env.run()
```

08_Stop_and_Wait_ARQ — Stop-and-Wait ARQ

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6208

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Stop-and-Wait receiver ready")
    conn, addr = server.accept()
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            frame = data.decode()
            print("Received:", frame)
            conn.sendall(("ACK:" + frame).encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6208
MAX_RETRIES = 3

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.settimeout(2)
    client.connect((HOST, PORT))
    for seq in range(1, 6):
        frame = f"FRAME-{seq}"
        for attempt in range(1, MAX_RETRIES + 1):
            try:
                print("Sending", frame, "attempt", attempt)
                client.sendall(frame.encode())
                ack = client.recv(1024).decode()
                if ack == "ACK:" + frame:
                    print("Received", ack)
                    break
            except socket.timeout:
                print("Timeout; retrying")
        else:
            print("Dropped", frame)
```

simpy_simulation.py

```
import simpy
import random

random.seed(13)
def sender(env, frames, channel):
    for frame in range(1, frames + 1):
        attempts = 0
        while attempts < 4:
            attempts += 1
            with channel.request() as req:
                yield req
                print(f"{env.now:.1f}: Send frame {frame}, attempt {attempts}")
                yield env.timeout(0.5)
            if random.random() > 0.25:
                print(f"{env.now:.1f}: ACK {frame} received")
                break
            print(f"{env.now:.1f}: ACK timeout")
        else:
            print("Frame dropped:", frame)

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(sender(env, 8, channel))
env.run()
```

09_Go_Back_N — Go-Back-N ARQ

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6209

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("GBN receiver demo ready")
    conn, addr = server.accept()
    with conn:
        expected = 1
        while True:
            data = conn.recv(1024)
            if not data:
                break
            try:
                seq = int(data.decode().strip())
            except ValueError:
                continue
            if seq == expected:
                print("Accepted frame", seq)
                conn.sendall(f"ACK:{seq}".encode())
                expected += 1
            else:
                print("Discarded out-of-order frame", seq, "expected", expected)
                conn.sendall(f"ACK:{expected-1}".encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6209
WINDOW = 4
TOTAL = 8
LOST_ONCE = 3
lost_done = False

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    base = 1
    while base <= TOTAL:
        next_seq = min(base + WINDOW - 1, TOTAL)
        restart = False
        for seq in range(base, next_seq + 1):
            if seq == LOST_ONCE and not lost_done:
                print("Simulated loss of frame", seq)
                lost_done = True
                print("Go back and resend from", seq)
                base = seq
                restart = True
                break
            client.sendall(str(seq).encode())
            ack = client.recv(1024).decode()
            print("Sent", seq, "received", ack)
        if not restart:
            base = next_seq + 1
```

simpy_simulation.py

```
import simpy
import random

random.seed(14)
def gbn_sender(env, total, window, channel, lost_frame):
    base = 1
    lost_once = False
    while base <= total:
        end = min(base + window - 1, total)
        print(f"{env.now:.1f}: Send window {base}-{end}")
        failure = None
        for seq in range(base, end + 1):
            with channel.request() as req:
                yield req
                print(f"{env.now:.1f}: Send frame {seq}")
                yield env.timeout(0.2)
            if seq == lost_frame and not lost_once:
                lost_once = True
                failure = seq
                print(f"{env.now:.1f}: Frame {seq} lost")
                break
        if failure is not None:
            yield env.timeout(1)
            print(f"{env.now:.1f}: Timeout; retransmit from {failure}")
            base = failure
            lost_frame = -1
        else:
            yield env.timeout(0.3)
            print(f"{env.now:.1f}: Cumulative ACK {end}")
            base = end + 1

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(gbn_sender(env, 8, 4, channel, lost_frame=3))
env.run()
```

10_Selective_Repeat — Selective Repeat ARQ

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6210

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Selective Repeat receiver demo ready")
    conn, addr = server.accept()
    with conn:
        received = set()
        while True:
            data = conn.recv(1024)
            if not data:
                break
            try:
                seq = int(data.decode().strip())
            except ValueError:
                continue
            received.add(seq)
            print("Buffered frame", seq, "received frames:", sorted(received))
            conn.sendall(f"ACK:{seq}".encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6210
TOTAL = 6
LOST_ONCE = 3
lost_done = False

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    pending = set(range(1, TOTAL + 1))
    for seq in range(1, TOTAL + 1):
        if seq == LOST_ONCE and not lost_done:
            print("Simulated loss of frame", seq)
            lost_done = True
            continue
        client.sendall(str(seq).encode())
        ack = client.recv(1024).decode()
        print("Sent", seq, "received", ack)
        pending.discard(seq)
    for seq in sorted(pending):
        print("Retransmitting only missing frame", seq)
        client.sendall(str(seq).encode())
        print("Received", client.recv(1024).decode())
```

simpy_simulation.py

```
import simpy
import random

random.seed(15)
def selective_sender(env, total, window, channel, lost_frame):
    acked = set()
    lost_once = False
    for start in range(1, total + 1, window):
        end = min(start + window - 1, total)
        for seq in range(start, end + 1):
            with channel.request() as req:
                yield req
                print(f"{env.now:.1f}: Send frame {seq}")
                yield env.timeout(0.2)
            if seq == lost_frame and not lost_once:
                lost_once = True
                print(f"{env.now:.1f}: Frame {seq} lost")
            else:
                acked.add(seq)
                print(f"{env.now:.1f}: ACK {seq}")
        for seq in range(start, end + 1):
            if seq not in acked:
                yield env.timeout(0.5)
                print(f"{env.now:.1f}: Retransmit only frame {seq}")
                acked.add(seq)
                print(f"{env.now:.1f}: ACK {seq}")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(selective_sender(env, 8, 4, channel, lost_frame=3))
env.run()
```

11_Bit_Stuffing — Bit stuffing and destuffing

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6211

def destuff(bits):
    result = []
    ones = 0
    i = 0
    while i < len(bits):
        bit = bits[i]
        result.append(bit)
        if bit == "1":
            ones += 1
            if ones == 5:
                if i + 1 < len(bits) and bits[i + 1] == "0":
                    i += 1
                ones = 0
        else:
            ones = 0
        i += 1
    return "".join(result)

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Bit destuff server ready")
    while True:
        conn, addr = server.accept()
        with conn:
            bits = conn.recv(4096).decode().strip()
            result = destuff(bits)
            conn.sendall(result.encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6211

def stuff(bits):
    out = []
    ones = 0
    for bit in bits:
        if bit not in "01":
            raise ValueError("Input must contain only 0 and 1")
        out.append(bit)
        if bit == "1":
            ones += 1
            if ones == 5:
                out.append("0")
                ones = 0
        else:
            ones = 0
    return "".join(out)

bits = input("Enter bit string: ").strip()
stuffed = stuff(bits)
print("Stuffed:", stuffed)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    client.sendall(stuffed.encode())
    result = client.recv(4096).decode()
    print("Destuffed at server:", result)
```

simpy_simulation.py

```
import simpy

def transmit(env, name, bits, channel):
    with channel.request() as req:
        yield req
        print(f"{env.now:.1f}: {name} begins transmitting {bits}")
        yield env.timeout(len(bits) * 0.01)
        print(f"{env.now:.1f}: {name} transmission complete")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(transmit(env, "Sensor-A", "101111101", channel))
env.process(transmit(env, "Sensor-B", "11111000", channel))
env.run()
```

12_CSMA — CSMA

server.py

```
# Simple TCP endpoint for sending station outcomes.
import socket

HOST = "127.0.0.1"
PORT = 6212
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("CSMA result receiver ready")
    while True:
        conn, addr = server.accept()
        with conn:
            data = conn.recv(1024).decode()
            print("Station result:", data)
            conn.sendall(b"Result received")
```

client.py

```
# One station's simplified carrier-sense decision.
import socket
import random

HOST = "127.0.0.1"
PORT = 6212
channel_busy = input("Is channel busy? (y/n): ").strip().lower() == "y"
if channel_busy:
    backoff = random.randint(1, 5)
    result = f"Channel busy; wait {backoff} slots before retry"
else:
    result = "Channel idle; transmit packet"
print(result)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    client.sendall(result.encode())
    print("Server:", client.recv(1024).decode())
```

simpy_simulation.py

Simplified carrier sensing; a single SimPy Resource serializes transmissions and does not model real overlapping radio collisions.

```
import simpy
import random

random.seed(16)
def station(env, sid, channel, stats):
    for packet in range(3):
        yield env.timeout(random.uniform(0.1, 1.0))
        if channel.count > 0:
            stats["busy"] += 1
            backoff = random.randint(1, 3)
            print(f"{env.now:.2f}: Station {sid} senses BUSY; backoff {backoff}")
            yield env.timeout(backoff)
        with channel.request() as req:
            yield req
            print(f"{env.now:.2f}: Station {sid} transmits")
            yield env.timeout(0.5)
            stats["success"] += 1
            print(f"{env.now:.2f}: Station {sid} done")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
stats = {"success": 0, "busy": 0}
for sid in range(1, 5):
    env.process(station(env, sid, channel, stats))
env.run()
print("Successful transmissions:", stats["success"])
print("Busy-channel deferrals:", stats["busy"])
```

13_CSMA_CA — CSMA/CA

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6213
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("CSMA/CA ACK server ready")
    while True:
        conn, addr = server.accept()
        with conn:
            packet = conn.recv(1024).decode()
            print("Received:", packet)
            conn.sendall(("ACK:" + packet).encode())
```

client.py

```
import socket
import random
import time

HOST = "127.0.0.1"
PORT = 6213
CW_MIN, CW_MAX, MAX_RETRIES = 4, 16, 4

for packet_no in range(1, 6):
    cw = CW_MIN
    delivered = False
    for attempt in range(MAX_RETRIES + 1):
        backoff = random.randint(0, cw - 1)
        print(f"Packet {packet_no}: backoff {backoff} slots")
        time.sleep(backoff * 0.05)
        try:
            with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
                client.settimeout(2)
                client.connect((HOST, PORT))
                payload = f"PACKET-{packet_no}"
                client.sendall(payload.encode())
                ack = client.recv(1024).decode()
            print("Received", ack)
            delivered = True
            break
        except (OSError, socket.timeout) as e:
            print("Failure/timeout:", e)
            cw = min(cw * 2, CW_MAX)
    if not delivered:
        print("Packet dropped after retries")
```

simpy_simulation.py

Simplified backoff model, not a full 802.11 MAC.

```
import simpy
import random

random.seed(17)
def station(env, sid, channel, stats):
    cw_min, cw_max, max_retries = 4, 16, 4
    for packet in range(2):
        cw = cw_min
        for attempt in range(max_retries + 1):
            backoff = random.randint(0, cw - 1)
            yield env.timeout(backoff)
            if channel.count > 0:
                stats["busy"] += 1
                cw = min(cw * 2, cw_max)
                print(f"{env.now:.1f}: S{sid} busy; CW={cw}")
                continue
            with channel.request() as req:
                yield req
                print(f"{env.now:.1f}: S{sid} transmits packet {packet+1}")
                yield env.timeout(1)
            stats["success"] += 1
            break
        else:
            stats["dropped"] += 1

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
stats = {"success": 0, "busy": 0, "dropped": 0}
for sid in range(1, 5):
    env.process(station(env, sid, channel, stats))
env.run()
print(stats)
```

14_Packet_Routing — Packet routing

server.py

```
import socket
import ipaddress

HOST = "127.0.0.1"
PORT = 6214
ROUTES = [
    ("192.168.1.0/24", "192.168.1.1", "eth0"),
    ("10.0.0.0/8", "10.0.0.1", "eth1"),
    ("0.0.0.0/0", "192.168.1.1", "eth0"),
]

def route(destination):
    ip = ipaddress.ip_address(destination)
    matches = []
    for network_text, next_hop, interface in ROUTES:
        network = ipaddress.ip_network(network_text)
        if ip in network:
            matches.append((network.prefixlen, network_text, next_hop, interface))
    if not matches:
        return "NO_ROUTE"
    _, network, next_hop, interface = max(matches)
    return f"network={network};next_hop={next_hop};interface={interface}"

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Static routing server ready")
    while True:
        conn, addr = server.accept()
        with conn:
            destination = conn.recv(1024).decode().strip()
            try:
                reply = route(destination)
            except ValueError:
                reply = "INVALID_IP"
            conn.sendall(reply.encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6214

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    destination = input("Enter destination IPv4 address: ").strip()
    client.sendall(destination.encode())
    print("Routing decision:", client.recv(1024).decode())
```

simpy_simulation.py

```
import simpy
import random

random.seed(18)
def packet(env, pid, router, stats):
    start = env.now
    yield env.timeout(random.uniform(0, 1))
    with router.request() as req:
        yield req
        print(f"{env.now:.2f}: Router processes packet {pid}")
        yield env.timeout(0.2)
    stats["forwarded"] += 1
    stats["delay"] += env.now - start

env = simpy.Environment()
router = simpy.Resource(env, capacity=1)
stats = {"forwarded": 0, "delay": 0.0}
for pid in range(1, 11):
    env.process(packet(env, pid, router, stats))
env.run()
print("Forwarded:", stats["forwarded"])
print("Average delay:", round(stats["delay"] / stats["forwarded"], 3))
```

15_Subnetting — Subnetting

server.py

```
import socket
import ipaddress

HOST = "127.0.0.1"
PORT = 6215
def calculate(cidr):
    net = ipaddress.ip_network(cidr, strict=False)
    first = str(next(net.hosts())) if net.num_addresses > 2 else "N/A"
    hosts = list(net.hosts()) if net.num_addresses <= 65536 and net.num_addresses > 2 else []
    last = str(hosts[-1]) if hosts else "N/A"
    return f"network={net.network_address};broadcast={net.broadcast_address};prefix={net.prefixlen};first={first};last={last}"

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Subnet calculator server ready")
    while True:
        conn, addr = server.accept()
        with conn:
            cidr = conn.recv(1024).decode().strip()
            try:
                reply = calculate(cidr)
            except ValueError:
                reply = "INVALID_CIDR"
            conn.sendall(reply.encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6215
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    cidr = input("Enter CIDR (e.g. 192.168.1.0/24): ")
    client.sendall(cidr.encode())
    print(client.recv(2048).decode())
```

simpy_simulation.py

```
import simpy

def calculation(env, cidr, cpu):
    start = env.now
    with cpu.request() as req:
        yield req
        print(f"{env.now:.1f}: Calculate subnet for {cidr}")
        yield env.timeout(0.2)
    print(f"Calculation time: {env.now-start:.1f}")

env = simpy.Environment()
cpu = simpy.Resource(env, capacity=1)
for cidr in ["192.168.1.0/24", "10.0.0.0/8", "172.16.0.0/16"]:
    env.process(calculation(env, cidr, cpu))
env.run()
```

16_Flow_Control — Flow control

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6216
WINDOW = 3

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Flow-control receiver ready")
    conn, addr = server.accept()
    with conn:
        expected = 1
        while True:
            data = conn.recv(1024)
            if not data:
                break
            seq = int(data.decode().strip())
            print("Received frame", seq)
            conn.sendall(f"ACK:{seq};WINDOW:{WINDOW}".encode())
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6216
TOTAL = 10
window = 3
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    seq = 1
    while seq <= TOTAL:
        end = min(seq + window - 1, TOTAL)
        for frame in range(seq, end + 1):
            client.sendall(str(frame).encode())
            ack = client.recv(1024).decode()
            print("Sent", frame, "server:", ack)
        seq = end + 1
```

simpy_simulation.py

```
import simpy

def sender(env, total, window, link):
    for start in range(1, total + 1, window):
        batch = range(start, min(start + window, total + 1))
        print(f"{env.now:.1f}: send window {list(batch)}")
        for frame in batch:
            with link.request() as req:
                yield req
                yield env.timeout(0.2)
                print(f"{env.now:.1f}: ACK frame {frame}")
        yield env.timeout(0.1)

env = simpy.Environment()
link = simpy.Resource(env, capacity=1)
env.process(sender(env, 10, 3, link))
env.run()
```

17_Congestion_Control — Congestion control

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6217
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Congestion demo receiver ready")
    conn, addr = server.accept()
    with conn:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print("Received segment:", data.decode())
            conn.sendall(b"ACK")
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6217
threshold = 8
cwnd = 1

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    for round_no in range(1, 9):
        print(f"Round {round_no}: cwnd={cwnd}")
        for segment in range(cwnd):
            client.sendall(f"round{round_no}-segment{segment+1}".encode())
            client.recv(1024)
        if cwnd < threshold:
            cwnd *= 2
        else:
            cwnd += 1
```

simpy_simulation.py

```
import simpy

def tcp_flow(env, rounds, threshold):
    cwnd = 1
    for r in range(1, rounds + 1):
        print(f"{env.now:.1f}: round {r}, cwnd={cwnd}")
        yield env.timeout(1)
        cwnd = cwnd * 2 if cwnd < threshold else cwnd + 1

env = simpy.Environment()
env.process(tcp_flow(env, 8, 8))
env.run()
```

18_Network_Performance — Network performance

server.py

```
import socket
import time

HOST = "127.0.0.1"
PORT = 6218
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("Performance receiver ready")
    conn, addr = server.accept()
    with conn:
        count = 0
        start = time.perf_counter()
        while True:
            data = conn.recv(1024)
            if not data:
                break
            count += 1
            conn.sendall(b"ACK")
        duration = time.perf_counter() - start
        print("Packets/messages:", count)
        print("Duration:", round(duration, 4), "seconds")
        print("Throughput:", round(count / duration, 2) if duration else 0, "messages/sec")
```

client.py

```
import socket
import time

HOST = "127.0.0.1"
PORT = 6218
TOTAL = 20
start = time.perf_counter()
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    for i in range(TOTAL):
        client.sendall(f"packet-{i+1}".encode())
        client.recv(1024)
duration = time.perf_counter() - start
print("Sent:", TOTAL)
print("Elapsed seconds:", round(duration, 4))
print("Throughput:", round(TOTAL / duration, 2) if duration else 0, "messages/sec")
```

simpy_simulation.py

```
import simpy
import random

random.seed(19)
def packet(env, pid, channel, stats):
    start = env.now
    yield env.timeout(random.uniform(0.1, 1.0))
    with channel.request() as req:
        yield req
        yield env.timeout(0.1)
    stats["delivered"] += 1
    stats["delay"] += env.now - start

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
stats = {"delivered": 0, "delay": 0.0}
for pid in range(1, 21):
    env.process(packet(env, pid, channel, stats))
env.run()
print("Delivered:", stats["delivered"])
print("Average delay:", round(stats["delay"] / stats["delivered"], 3))
print("Throughput:", round(stats["delivered"] / env.now, 3), "packets/time-unit")
```

19_Wireless_IoT — Wireless IoT

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6219
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as server:
    server.bind((HOST, PORT))
    print("IoT sensor UDP receiver ready")
    while True:
        data, addr = server.recvfrom(1024)
        print("Sensor", addr, "sent:", data.decode())
        server.sendto(b"ACK", addr)
```

client.py

```
import socket
import random
import time

HOST = "127.0.0.1"
PORT = 6219
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as client:
    client.settimeout(2)
    for sensor_id in range(1, 6):
        temp = round(random.uniform(20, 40), 2)
        packet = f"sensor={sensor_id};temperature={temp}"
        client.sendto(packet.encode(), (HOST, PORT))
        try:
            print("Server:", client.recvfrom(1024)[0].decode())
        except socket.timeout:
            print("No ACK for sensor", sensor_id)
        time.sleep(0.2)
```

simpy_simulation.py

```
import simpy
import random

random.seed(20)
def sensor(env, sid, channel, stats):
    while True:
        yield env.timeout(random.uniform(1, 3))
        value = round(random.uniform(20, 40), 1)
        with channel.request() as req:
            yield req
            yield env.timeout(0.1)
        stats["sent"] += 1
        print(f"{env.now:.1f}: Sensor {sid} sends {value} C")
        if value > 35:
            stats["alerts"] += 1
            print("ALERT: high temperature")

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
stats = {"sent": 0, "alerts": 0}
for sid in range(1, 5):
    env.process(sensor(env, sid, channel, stats))
env.run(until=10)
print(stats)
```

20_CDMA — CDMA

server.py

```
import socket

HOST = "127.0.0.1"
PORT = 6220
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, PORT))
    server.listen()
    print("CDMA demo receiver ready")
    conn, addr = server.accept()
    with conn:
        data = conn.recv(4096).decode()
        print("Combined chip signal:", data)
        conn.sendall(b"Signal received")
```

client.py

```
import socket

HOST = "127.0.0.1"
PORT = 6220
# Two example orthogonal chipping codes
codes = {"A": [1, 1, 1, 1], "B": [1, -1, 1, -1]}
bits = {"A": 1, "B": 0}  # map 1 -> +1, 0 -> -1

combined = [0] * 4
for user, bit in bits.items():
    value = 1 if bit == 1 else -1
    for i, chip in enumerate(codes[user]):
        combined[i] += value * chip

print("Combined signal:", combined)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client:
    client.connect((HOST, PORT))
    client.sendall(str(combined).encode())
    print("Server:", client.recv(1024).decode())

# Decode user A locally by correlation:
a_value = sum(combined[i] * codes["A"][i] for i in range(4)) // 4
print("Decoded A bit:", 1 if a_value > 0 else 0)
```

simpy_simulation.py

The client demonstrates a small CDMA code/correlation example. A fully realistic CDMA physical-layer simulation is beyond this socket demo.

```
import simpy

def user_transmit(env, user, code, channel):
    with channel.request() as req:
        yield req
        print(f"{env.now:.1f}: User {user} transmits chips {code}")
        yield env.timeout(1)

env = simpy.Environment()
channel = simpy.Resource(env, capacity=1)
env.process(user_transmit(env, "A", [1, 1, 1, 1], channel))
env.process(user_transmit(env, "B", [1, -1, 1, -1], channel))
env.run()
```